Compiladores e Intérpretes

Tema 4.3 – Analizador Semántico – Tabla(s) de Símbolos

Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

2026

Intuiciones Análisis Semántico

- Las tareas del **analizador semántico**, en general para un lenguaje de programación con **tipado estático**, se pueden dividir en **dos**:

Entender y controlar todas las entidades declaradas
Chequeo de Declaraciones

Entender y controlar todas las sentencias asociadas a las entidades recolectadas
Chequeo de Sentencias

Esquemas de Traducción

Producción	Reglas Semanánticas
D → T L	L.tipoH = T.tipo
T → **idC**	T.tipo = *tipoClase*(**idC**.*lex*)
T → **String**	T.tipo = *tipoString*
T → **int**	T.tipo = *tipoInt*
L → **idMV** R	Guardar(**idMV**.*lex*, L.tipoH) R.tipoH = L.tipoH
R_1 → **idMV** R_2	Guardar(**idMV**.*lex*, R_1.tipoH) R_2 .tipoH = R_1.tipoH
R → e	{}

Esquema de Traducción (EDT)
S → D$
D → T {L.tipoH = T.tipo} L
T → **idC** {T.tipo = *tipoClase*(**idC**.*lex*)}
T → **String** {T.tipo = *tipoString*}
T → **int** {T.tipo = *tipoInt*}
L → **idMV** {R.tipoH = L.tipoH} R {Guardar(**idMV**.*lex*, L.tipoH)}
R_1 → **idMV** {R_2.tipoH = R_1.tipoH} R_2 {Guardar(**idMV**.*lex*, R_1.tipoH)}
R → e

EDTs en Analizado

Esquema de Traducción (EDT)
S → D$
D → T {L.tipoH = T.tipo} L
T → **idC** {T.tipo = *tipoClase*(**idC**.*lex*)}
T → **String** {T.tipo = *tipoString*}
T → **int** {T.tipo = *tipoInt*}
L → **idMV** {R.tipoH = L.tipoH} R {Guardar(**idMV**.*lex*, L.tipoH)}
R_1 → **idMV** {R_2.tipoH = R_1.tipoH} R_2 {Guardar(**idMV**.*lex*, R_1.tipoH)}
R → e

```
void D()
    Type t = T()
    L(t)

Type T()
    if(tkAct es idC)
        String nom = tkAct.lex
        match("idC")
        return new TipoClase(nom)
    else if(tkAct es int)
        match("int")
        return new TipoInt()
    else if(tkAct es String)
        match("String")
        return new TipoInt()
    else ERROR SINTACTICO!

void L(Type tipo)
    String nom = tkAct.lex
    match("idMV")
    R(tipo)
    Guardar(nom, tipo)

void R(Type tipo)
    if(tkAct es idMV)
        String nom = tkAct.lex
        match("idMV")
        R(tipo)
        Guardar(nom, tipo)
    else if(tkActual es $)
    else ERROR SINTACTICO
```

Recopilando Información de las Entidades

- En general en los lenguajes es posible **declarar** varios tipos de **entidades**: variables, constantes, procedimientos, funciones, parámetros, estructuras, tipos, clases, etc.
- **Cada** tipo de **entidad** tiene sus **características propias** y su comportamiento, los cuales indican como:
  - **Usarlas** de forma correcta
  - **Traducirlas** de forma correcta

Recopilando Información de las Entidades

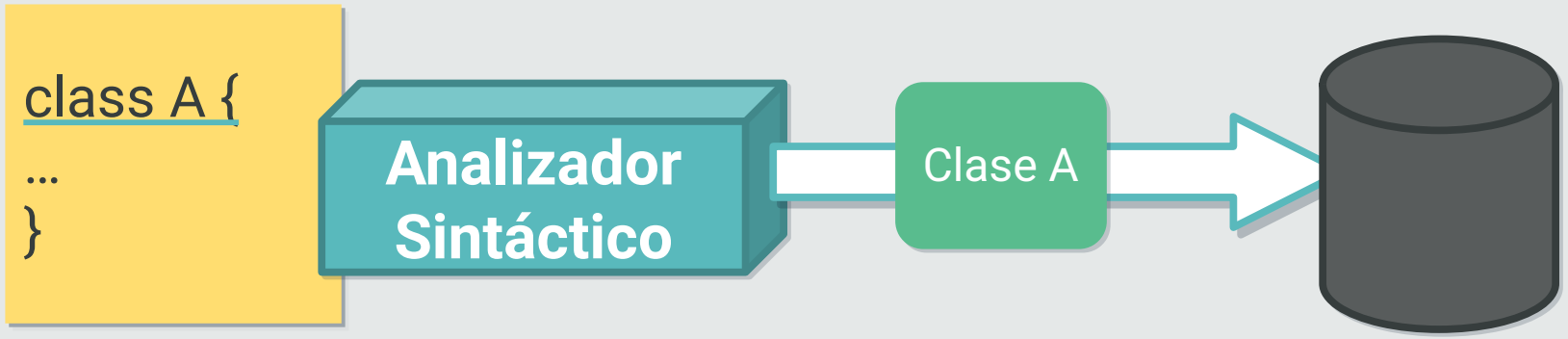
- A cada **entidad** declarada la **identificamos** con un **nombre** (usando un identificador)
- Por ejemplo: la clase A, el método m1, la variable v1
- Luego en el **programa** podemos hacer **uso de los nombres** de las **entidades declaradas** para:
  - **Declarar** nuevas entidades
  - Realizar **computaciones** (asignaciones, cálculos, llamadas, etc.)

Recopilando Información de las Entidades

- Una de las principales actividades de un compilador es **recolectar la información** de las **entidades declaradas** para, posteriormente cuando estas sean usadas:
  - **Resolver** con que **entidad** se **corresponde** un **identificador**
  - **Controlar** que la entidad identificada sea **correctamente usada**
  - Realizar una **traducción adecuada** según la entidad que se esta identificando

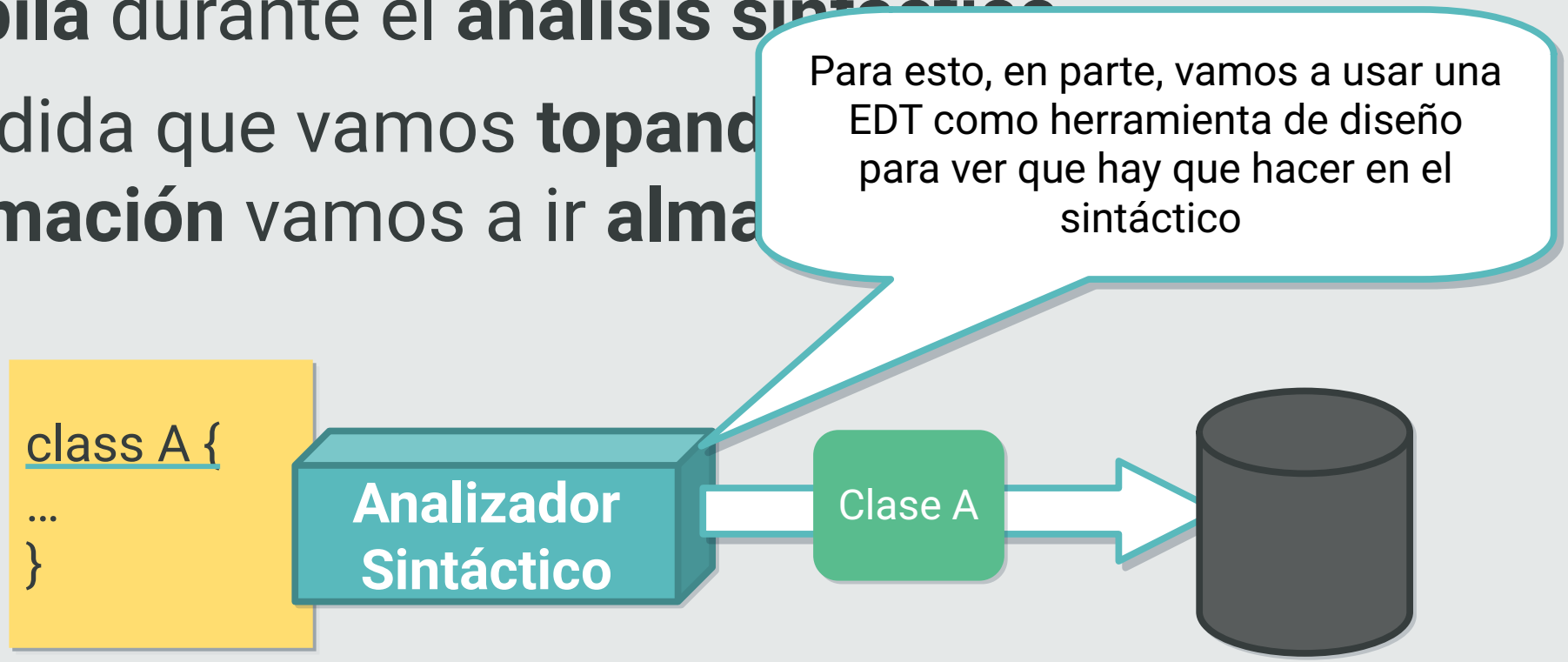
Recopilando Información de las Entidades

- Gran parte de la **información** de las **entidades declaradas** se **recopila** durante el **análisis sintáctico**
- A medida que vamos **topando** nos con las **entidades** y su **información** vamos a ir **almacenandolas**



Recopilando Información de las Entidades

- Gran parte de la **información** de las **entidades declaradas** se **recopila** durante el **análisis sint**
- A medida que vamos **topand** y su **información** vamos a ir **alma**



Recopilando Información de las Entidades

- La idea es tener un **repositorio global** a la EDT donde se vaya almacenando la **información de las entidades** a medida que se va **reconociendo sintácticamente** el programa
- Que, además, en caso de ser necesario pueda **recibirla el modulo** encargado de los **controles semánticos**

Tabla de Símbolos

Tabla de Símbolos

- La **tabla(s) de símbolos (TS)** es una (o mas) estructura(s) central(es) para el análisis semántico
- Mantiene **información** de todas las **entidades declaradas** (clases, métodos, variables, etc.)
- Cada **entidad** estará asociada a una **entrada**
- Una **entrada** tendrá toda la **información** del tipo de **entidad que representa**, tanto para el **análisis semántico** como para la **generación de código**

Tabla de Símbolos

- La función principal de TS es **buscar** y/o obtener la entrada de una **entidad** en base a un **nombre**
- La **tabla de símbolos**,
  - Debe ser **simple** de crear y administrar
  - Debe ser **funcional/útil** al análisis semántico y la traducción
  - Debe funcionar de forma **eficiente** ya que casi todos los aspectos del análisis semántico y la traducción refieren a la TS

Estructura de la Tabla de Símbolos

- ¿Cómo es la **estructura** de la **Tabla de Símbolos**?
- Una **tabla plana** con todos los nombres **no es adecuada**…

Ciertos nombres sólo son visibles/accesibles en ciertos contextos

```
class A{
    int x;
   void m1(){ … }
}
class B{
    void m1(){ … }
}
```

Es posible referenciar directamente a la variable de instancia x

No es posible referenciar directamente a la variable de instancia x

Estructura de la Tabla de Símbolos

- ¿Cómo es la **estructura** de la **Tabla de Símbolos**?
- Una **tabla plana** con todos los nombres **no es adecuada**…

Ciertos nombres sólo son visibles/accesibles en ciertos contextos

Un nombre puede representar entidades diferentes en diferentes contextos.

```
class A{
    boolean x;
    void m1(){ … }
    void m2(int x){ … }
}
```

En este método el nombre **x** corresponde a la **variable de instancia**

En este método el nombre **x** corresponde al **parámetro**

Estructura de la Tabla de Símbolos

- Para resolver esto vamos a **Anidar/Jerarquizar** adecuadamente **tablas** con las entidades correspondientes a cada **nivel**!
- En general, debemos estructurar la tabla de símbolos de manera tal que en un **contexto**, como el cuerpo de un método, sea claro a qué **nombres** se puede **referenciar**.
- Buscaremos que la estructura de la tabla respete lo mejor posible los **ambientes** de declaración determinados por la **sintaxis del lenguaje**

Estructura de la Tabla de Símbolos

```
class A {
   int x;
   B y;
   int m1(){…}
   void m2(String z){…}
}

class B extends A{
   int v;
   int m1(){…}
}
```

Estructura y Creación de la Tabla de Símbolos: MiniJava

Estructura de la Tabla de Símbolos

- La TS se va **creando** a medida que se **realiza el análisis sintáctico** (cuando procesamos la declaración de una entidad)
- Se diseña con **acciones semánticas** en la **EDT** y se trabaja a la **TS** como un **elemento global**
- La **estructura** debe provee **funciones** para poder **insertar una entrada** con un **nombre** en la tabla del **contexto** donde esta **declarada esa entidad**.

Estructura Tabla de Símbolos en MiniJava

- Cada tipo de **entrada** (clase, método, variable, etc.) se puede **representar** mediante una **clase**
- Las **variables de instancia** de la clase representan los diferentes **atributos** que caracterizan a la **entidad** asociada a la entrada.



Estructura Tabla de Símbolos en MiniJava

- ¿Cómo estructuramos las entradas de la TS de un programa MiniJava?
- La estructura **principal** es una **tabla de clases**
- Cada entrada de **clase** tiene una **tabla** de **vars** de instancia y una de **métodos**
- Cada entrada de **método/constructor** tiene una **tabla de parámetros**

TS

```
Hash<Clase>
…
```

Clase

```
Hash<Atributo> atributos
Hash<Metodo> metodos
…
```

Metodo

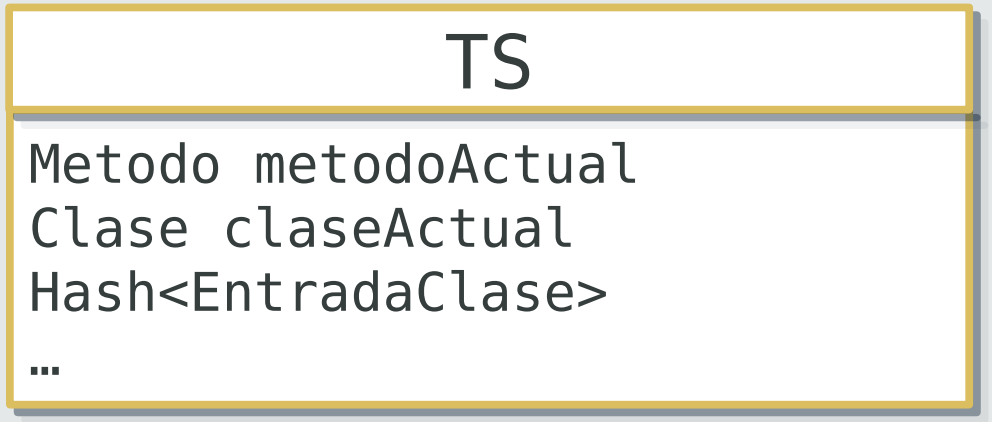
```
Tipo TipoRetorno
Hash<Parametro> params
…
```

Estructura Tabla de Símbolos en MiniJava

- Además las clases que representan las entradas, deben proveer **funciones** para **consultar/administrar** la tabla de símbolos y las distintas **entradas**.
- Por ejemplo:
  - La tabla de símbolos debería tener una **función** para ver si cierto **nombre** es **una clase declarada** y **devolver su entrada**
  - Una **entrada de clase** debería tener una **función** para ver si determinado **nombre** es un **método suyo**, y **devolver** la correspondiente **entrada de método**.

Estructura Tabla de Símbolos en MiniJava

- La **TS** debe proveer una **forma de identificar** cual es el **ambiente actual**



- ¿Por qué es importante?

Creando la Tabla de Símbolos en MiniJava

- La **tabla de símbolos** se creará a medida que se va **realizando** el **Análisis Sintáctico**.
- Haremos una **EDT** sobre la gramática utilizada en el Analizador Sintáctico, donde las **acciones semánticas construirán** la TS
- Luego, modificaremos los **métodos** del **Analizador Sintáctico** para reflejar las **acciones semánticas** de la EDT y así crear la **TS** que será utilizada en el análisis semántico

Creando la Tabla de Símbolos en MiniJava

- Consideremos la reglas

  <Clase> ::= **class idC** <Herencia> { <Miembros> }

  <Herencia> ::= **extends idC** | **e**

Declaración de clase:
¿Qué debemos hacer?



Dos estrategias:

- **Delegando** la inserción de los elementos en profundidad



- **Retornando** el/los elementos a insertar



Creando la Tabla de Símbolos en MiniJava

- Consideremos la reglas

  <Clase> ::= **class idC** <Herencia> { <Miembros> }

  <Herencia> ::= **extends idC** | **e**

Declaración de clase: ¿Qué debemos hacer?

Dos estrategias:

- **Delegando** la inserción de los elementos en profundidad
- **Retornando** el/los elementos a insertar





La idea es que en cada **producción inserto el elemento** vinculado a esa producción en la estructura correspondiente de la **TS**

Requiere llevar cuenta de la ClaseActual, Metodo ctual, etc.

Creando la Tabla de Símbolos en MiniJava

- Consideremos la reglas

  <Clase> ::= **class idC** <Herencia> { <Miembros> }

  <Herencia> ::= **extends idC** | **e**

Declaración de clase:
¿Qué debemos hacer?

Dos estrategias:

- **Delegando** la inserción de los elementos en profundidad
- **Retornando** el/los elementos a insertar





La idea es que en cada **producción retorno el elemento** vinculado a esa producción. Es responsabilidad de quien contiene ese elemento

Creando la Tabla de Símbolos en MiniJava

- Consideremos las reglas (version simplificada de Minijava):
  - <Miembro> ::= <AtributoMetodo><Miembro> | **e**
  - <AtributoMetodo> ::= <Tipo> **idMV** RestAoM
  - <RestAoM> ::= <Parametros> | **;**
  -



Declaración de atributos y métodos: ¿Qué debemos hacer?

Creando la Tabla de Símbolos en MiniJava

Una consideración importante con respecto al diseño de la TS es cómo se representarán los tipos



Creando la Tabla de Símbolos en MiniJava

Una consideración importante con respecto al diseño de la TS es cómo se representarán los tipos

Tipo Primitivo VS Tipo Referencia

Son **diferentes semánticamente** y por lo tanto deben modelarse por **separado**
(los tipos primitivos **NO** son clases)

EntradaVar

```
Tipo tipoVar
String nombre
…
```

Creando la Tabla de Símbolos en MiniJava

Una consideración importante con respecto al diseño de la TS es cómo se representarán los tipos

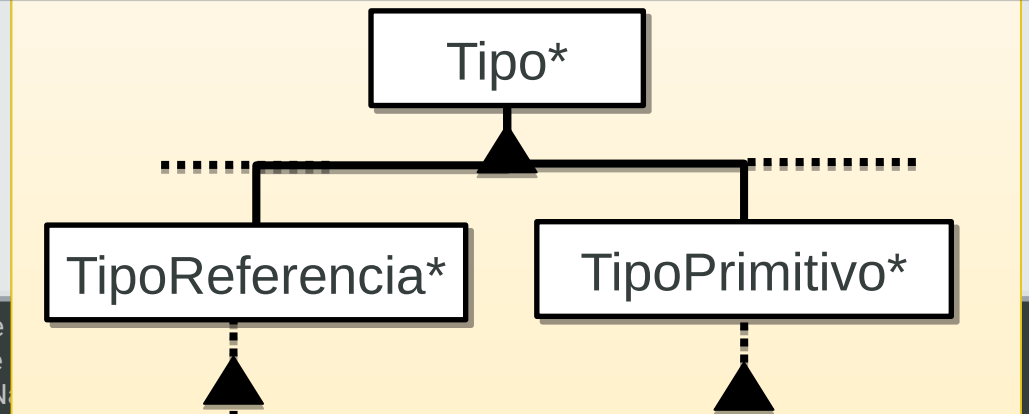
Tipo Primitivo VS Tipo Referencia

Son **diferentes semánticamente** y por lo tanto deben modelarse por **separado**
(los tipos primitivos **NO** son clases)

Por otra parte es importante **modelarlos** de manera **uniforme** para diseñar adecuadamente las **entidades que los utilizan** (métodos y variables)

EntradaVar

```
Tipo tipoVar
String nombre
…
```



Estructura de la Tabla de Símbolos

- En algunos lenguajes hay partes de la tabla que se **terminan de consolidar** en el **chequeo de declaraciones**
  - Vinculado a **entidades** que **dependen** de otras **entidades** que se declaran **mas adelante** (referencias adelante).
- Por ejemplo, en los lenguajes orientados a objetos cuando tenemos **herencia** no conocemos todos los métodos que tiene una clase hasta que **no procesamos a sus ancestros**.
  - En estos lenguajes una vez **finalizado el análisis sintáctico** se **consolidan las tablas** de métodos de las clases